DevPilot AI Project Guide

Architecture, RAG, agents, MCP tools, workspace safety, and evaluation

Purpose: This document is a sample knowledge source for the DevPilot AI RAG ingestion pipeline. It is written with headings and concise sections so the service can extract text, create chunks, embed them, and retrieve relevant passages.

1. Project Purpose

DevPilot AI is a software engineering assistant designed to help developers understand a project, retrieve relevant technical knowledge, propose code changes, run tests, and produce documentation. It should be treated as an assistant with controlled capabilities, not as an unrestricted shell operator.

2. High-Level Architecture

Component	Primary responsibility
React frontend	Login, dashboard, task submission, progress, results
API Gateway	Routing, token validation, CORS, rate limits, resilience
Auth Service	Registration, password hashing, login, JWT issuance
Eureka Server	Service registry and discovery
AI Orchestrator	Coordinates workflow, task state, agent sequencing
Knowledge Service	PDF ingestion, embeddings, retrieval, grounded answers
Developer MCP Server	Exposes explicitly allowed workspace tools
PostgreSQL + pgvector	Stores application data and vector embeddings

Service discovery is not a request-processing layer. The Gateway routes external API calls; Eureka helps services discover registered instances. MCP is a protocol for exposing tools to a client, not a mechanism for agent-to-agent messaging.

3. Example Software Engineering Workflow

- User submits a task such as adding a validated expense endpoint.
- Orchestrator creates a task record and defines allowed workspace scope.
- Research Agent searches the knowledge base and inspects relevant project files.
- Coding Agent proposes or applies a bounded change using approved tools.
- Testing Agent runs an allowlisted test command and captures the result.
- If tests fail, the orchestrator permits a small, bounded repair cycle.
- Documentation Agent records the change, test result, and relevant usage notes.
- The system returns a summary, changed-file list, and test evidence to the user.

4. RAG Knowledge Pipeline

Predefined knowledge documents and authorized user uploads pass through PDF text extraction, heading-aware chunking, embedding generation, and storage in PostgreSQL with pgvector. At query time, the system may rewrite a follow-up question into a standalone search query, retrieve relevant chunks, and ask a language model to answer using those chunks.

Use the same compatible embedding model for indexing and query embeddings. Retrieval quality depends on document quality, chunk boundaries, embedding model, filters, top-k selection, and relevance thresholds. A retrieved

passage is evidence, not a guarantee of correctness.

5. Tool Calling and MCP

Tool calling allows a model-driven workflow to request a named operation with structured arguments. The application must validate those arguments, authorize the operation, execute it, and return a bounded result. The model should not be trusted to choose arbitrary file paths or shell commands.

MCP standardizes how a client discovers and calls tools exposed by an MCP server. The MCP server contains or delegates the tool implementation. A JSON configuration commonly tells the client how to launch or connect to a server; it does not itself implement read or write operations.

Useful initial tools are read-only: list approved files, read a file within the workspace, search source text, and inspect project metadata. Add write or test-execution tools only with path restrictions, command allowlists, size limits, timeouts, and audit logs.

6. Workspace Safety Rules

- Restrict file operations to a configured workspace root and normalize paths before access.
- Reject path traversal and symbolic-link escapes where relevant.
- Do not expose environment variables, credentials, private keys, or arbitrary files.
- Avoid unrestricted shell execution; allow only named commands and arguments.
- Limit output size, execution time, and repair attempts.
- Require confirmation or review before destructive or high-impact changes.
- Record which tool ran, the authorized scope, and its result.

7. Data and Service Boundaries

Use PostgreSQL relational tables for application records and pgvector for embedding similarity search. Keep service ownership clear: the Auth Service owns identity operations, Knowledge Service owns knowledge ingestion and retrieval, and the orchestrator owns task workflow state.

Never trust a userId supplied by the LLM or request body to authorize access. Derive the authenticated principal from validated security context and apply ownership filters in the backend.

8. Evaluation Metrics

Evaluate the project using functional correctness, authorization tests, retrieval quality, response grounding, latency, failure handling, and traceability. For retrieval, precision@k measures how many retrieved items are relevant among the top k; recall@k measures how many known relevant items were retrieved in the top k.

Track task success rate, test pass rate, repair attempts, retrieval latency, generation latency, and errors. Do not claim quality improvements without a defined test set and measured results.

9. Demo Questions for DevPilot RAG

- What is the role of the AI Orchestrator compared with the MCP Server?
- How does DevPilot turn a PDF into retrievable knowledge?
- Why must workspace tools enforce path restrictions?
- What is the difference between Eureka service discovery and API Gateway routing?
- How should the orchestrator limit automated code repair attempts?
- What evidence should the system show after running tests?

End of document. Use this file for development and testing; verify all examples against your actual application configuration before treating them as production standards.